Expression transfer on Face meshes through 2D monocular guidance

Francesco Giarrusso[1], Luca Del Signore[1]

[1]Department of Computer, Control, and Management Engineering Antonio Ruberti (DIAG), Sapienza University of Rome, Rome, Italy

Abstract

In the last decade, the demand for automatic tools for deforming meshes has been increasing, lead by technological advancements in fields such as health, art, manufactures and entertainment. Particularly in this last field, the ability to modify facial expressions on fictional characters is crucial for producing realistic and engaging content. We propose a dynamic, fully automatic and real-time framework for deforming a face mesh in order to match a desired expression extracted from a 2D image ("2D guidance"). Our technique extracts meaningful characteristics from target expression in the image, such as landmark position and feature embeddings. Through differentiable rendering, the 2D feedback coming from the rendered mesh and target image is projected into 3D space, ensuring the resulting mesh to match the target expression. In other words, this process learns the necessary displacements to be applied to each vertex of the source mesh to accurately reproduce the target expression. It is important to note that our method does not require a large dataset, but relies solely on a single target 2D face and the source mesh on which we want to imprint the expression.

Keywords

Face Mesh deformation, Deep learning, Expression transfer, Differentiable rasterization, 2D Guidance

Highlights

- Light-weight and real-time Face deforming process;
- Expression Transfer improved with respect to Baseline Approaches;
- Leveraging the characteristics and features that can be collected from a face image;
- Modifying and adapting, in a differentiable way, state-of-the-art frameworks for retrieving face properties;
- No Dataset needed.

1. Introduction

As a consequence of increasing power of GPUs, recent interest for simulated 3D environments and high demand of 3D games, many frameworks that involve deformations of 3D structures have been proposed; these models rely on representations of 3D structures through Point Cloud, Voxel Grid or most commonly, 3D Meshes. In this scenario remarkable examples of successful frameworks are Text2Mesh[1] and Pixel2Mesh[2], as they represent a benchmark for related works in the field. The former aims to deform a mesh to match the content of a given input text, on the other hand, the latter transfers features and style of an input photo to a source ellipsoid mesh. Although these frameworks are really general and powerful, they might be not sufficiently specialized to capture and modify fine-grained details required for some specific applications. A clear example is the deformation of a neutral face to match a desired facial expression, which is the topic of our project. The described task is preeminent in many applications of the entertainment industry, such as games, animation of movies and simulated environments requiring virtual reality, for instance to produce realistic avatars [3]. Similar approaches can also be found in rehabilitative and medical applications, for example, Ferrari et al. [4] deals with expression monitoring to keep track of the execution of rehabilitation exercises by patients suffering from Parkinson or Alzheimer.

In the existing literature, many ways to guide the mesh deformation are proposed. A possible choice is represented by displacing vertices of an existing source mesh (commonly an ellipsoid) to be in the same position of the ones from a provided target mesh; in Text2Mesh, as said, the input text is used as guidance. Another possible approach is to use a monocular image to guide the displacement, as well as pixel2Mesh; we decided to follow this path and use a 2D guidance, although, as far as we know, there are not many notable deep-learning frameworks that rely on 2D guidance for modifying fine-grained details on face meshes. Additionally, the chosen approach is more flexible, as a matter of facts, 3D spatial information is not always available. Other similar works employ traditional techniques or are not fully automated since some sort of user intervention is required.

Our framework combines and personalizes some tools and ideas from the state-of-the-art in 3D shape deformation field:

giarrusso.1807094@studenti.uniroma1.it (F. Giarrusso);
delsignore.2096442@studenti.uniroma1.it (L. D. Signore)
© 2021 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).
CEUR Workshop Proceedings (CEUR-WS.org)

1. Zero-shot learning idea of the Text2Mesh [1], so that a large dataset is not required;
2. The idea of VGG16 network feature extractor of Pixel2Mesh [2], to gain information on low-level and high-level features of the target image;
3. Pytorch3D module [5] for differentiable rendering and tools that allow the gradient flow from the rendered image to the 3D structure from which the photo was generated.

These considerations lead to the following base structure of our training process, inspired on the design idea of Text2Mesh, with these modifications:

- Instead of using Neural Style field with a positional encoder, we employed simpler neural architectures. The models learn vertex displacement by determining how much each vertex should move along the direction of the vector normal to the surface;
- We removed the CLIP Joint embedding space [6] as it is not capable of capturing the characteristic details of a target expression;
- Naturally followed from the second point, we have substituted the semantic loss with a combination of losses that employs the information taken from a face image (landmarks and features).

As implicitly stated in the last point, the core idea of our work is to extract relevant features from an image face, using them as feedback to guide the deformation made by the network. Specifically we aim to minimize, through loss function, the distance between features coming from the rendered view of the deformed mesh and those of the target expression image. A pretrained VGG16 [7] pytorch model is used to retrieve low-level and high-level feature maps. Additionally, facial keypoints (called also landmarks) are detected by adapting the existing framework [8]. Since this architecture mainly relies on not differentiable operations, we have modified it in order to allow gradient flow. We are not aware of any other notable framework that generates 3D displacement being guided by 2D differentiable landmark detection, thus it represents another original contribution of our work.

In particular, we have defined two different pipelines characterized by different losses and networks depending on the fact we want to transfer to our source mesh a 'Moderate Facial Expression' (MFE) or an 'Extreme facial expression' (EFE). A further distinction can be made according to the subject of the source mesh. When both source mesh and target image represent the same individual, the challenge shifts from expression transfer to align the source mesh to an hypothetical target mesh, through 2D guidance (mesh reconstruction). The target image, in this case, is the frontal rendering view of the target mesh, representing exactly that person with that particular target expression. For this reason, we decided to compare the performances on this specific task of our proposed networks against NSF of text2mesh and the default implementation of PyTorch3D. For the evaluation we relied on Chamfer, Hausdorff and Earth Mover distances [9, 10, 11]; however, when the source and target represent different individuals, these metrics become meaningless since they do not accurately differentiate expression modifications from variations in identity. As we will see in chapter 4, in the evaluated task our method reports consistently better results with respect to the compared approaches, both in terms of quality and computation time.

2. Related Works

In this section, we will explore researches related to this project, including both directly relevant researches from which we have taken inspiration and other state-of-the-art projects that deal with face reconstruction and facial expression transferring on 3D meshes.

2.1. Style and Content Transfer

The work of Johnson and al. [12] had a significant impact on computer vision tasks and paved the way for content and style transfer based on image. The underlying architecture takes as input the source image to be modified and a target image that contains the desired texture and pattern. It uses a style-loss in order to see if the wanted stylization matches that of the newly modified image generated by the network. Another component, called content loss, is used to ensure the consistency of the subject of the source image while inserting the style. We adapted the mentioned framework to our purposes by extending the content loss and, since we are not interested in the style, removing the style-loss.

2.2. Text2Mesh

Text2Mesh [1] is a framework designed to imprint a desired style to a target mesh, both geometrically and "texturally". Leveraging the CLIP joint embedding space [6], it allows the user to specify the desired stylilization both through text and image. At every iteration many views of the deformed mesh are generated through differentiable renderer, and for each of them CLIP embeddings are computed. In order to guide the process, "Semantic Loss" is employed: it is defined as the sum of the cosine similarities between each image-embedding and the embedding of the input text. Text2Mesh relies on Neural Style Field (NSF) Network, which is essentially a multilayer perceptron with ReLU activation functions between fully connected layers. At input layer, a positional

encoder [13] is employed to help the MLP in capturing high-frequency details. In Chapter 4 we attempted to replicate and adapt the NSF structure to our own approach, removing the color branch in order to have a fair comparison with our proposed networks.

2.3. Similar Works

In this chapter we provide an overview of some of the methodologies in the existing literature related to our work. We can differentiate them in three major families of solutions, based on the principal approach followed:

1. **Statistic approaches**: referred in literature as 3D Morphable models (3DMM), they are based on principal component analysys (PCA), for compact facial shapes representation. They learn, through statistical analisys on a huge number of 3D faces scans, principal components' parameters that are responsible for significant variations, enabling the manipulation of existing meshes and reconstruction of new ones. They have been heavily exploited, being capable of reconstructing plausible 3D models though without accounting for fine-grained personal details. There are some drawbacks of this kind of approach: these linear models are not capable of expressing and capturing non-linear features. Another limitation is represented the highly sensitiveness to variations in the input mesh. An example is the situation when the number of vertices differs from those of the meshes of the training dataset or when there is no point-to-point correspondence with the average measured mesh. Furthermore, some preprocessing and resampling operations are required, such as Mesh registration. Many works related to this topic use this strategy, but a notable example is [4].
2. **Geometric function approaches**: some of these works put a lot of effort on searching hardcoded geometric formulas to be optimized rather than using a learning-based network to estimate functions in a more general way. In fact, these approaches tend to be static and non-scalable, because they are limited to handling input mesh with some specific geometric features, as initially designed; sometimes some fine-tuning intervention is required. For example, [3] is heavily based on the alignment of landmarks. As a matter of facts, the user is requested to select the keypoint regions on a virtual avatar in order to make this process work; however, when a new avatar is used, a new selection process is necessary. This process can be time-consuming and can lead to errors if the landmarks are not positioned accurately. Moreover, it struggles to handle complex non-linear deformations and needs further manual optimization.
3. **Deep-Learning approaches**: most of related deep learning methods focus on the reconstruction of the face from a monocular image, as seen in [2], rather than on expression transfer. However, there are some approaches based on deformations in expression, but they work completely in 3D without relying on the guidance of a monocular image, trying to learn a latent space based on the encoder-decoder structure. Examples include [14] which tries to learn a noise pattern to be added to source meshes and [15] which relies on mesh operators for deformations.

Our proposed framework achieves the advantages of the methods mentioned above and mitigate their weakness, enlarging the scenario of deep-learning approaches: first of all it is fully automatic, it is independent from the input mesh type, moreover it can deal with fine-grained details and non-linear muscle lineaments. Additionally, it is real-time and does not require huge 3D high resolution faces dataset.

2.4. Differentiable rendering

Differentiable rendering [16] approximates and extends the rasterization and the shading process so that gradients can be back-propagated through the network. Allowing the gradient flow, the model can directly optimize the parameters of the network that produces displacement on 3D mesh through loss functions computed from 2D image. Unfortunately, the aforementioned pytorch3D implementation is non-deterministic and does not allow to specify a seed.

3. Proposed Framework

This section provides a deep description of our design choices, the modifications and extensions performed on pre-existing frameworks and exhaustive insights on the deformation pipelines that guide optimization of network parameters.

3.1. Base Strategy

As mentioned before, we took specific inspiration from Text2Mesh and Pixel2Mesh frameworks. As Text2Mesh, we followed a 'Zero-Shot Learning' approach, where the process of optimization of deformations is on the same source mesh. At every iteration, new deformations are added on the original mesh structure, to match the expression of 2D facial image, used as "guidance". In order evaluate how well the deformation of the mesh aligns

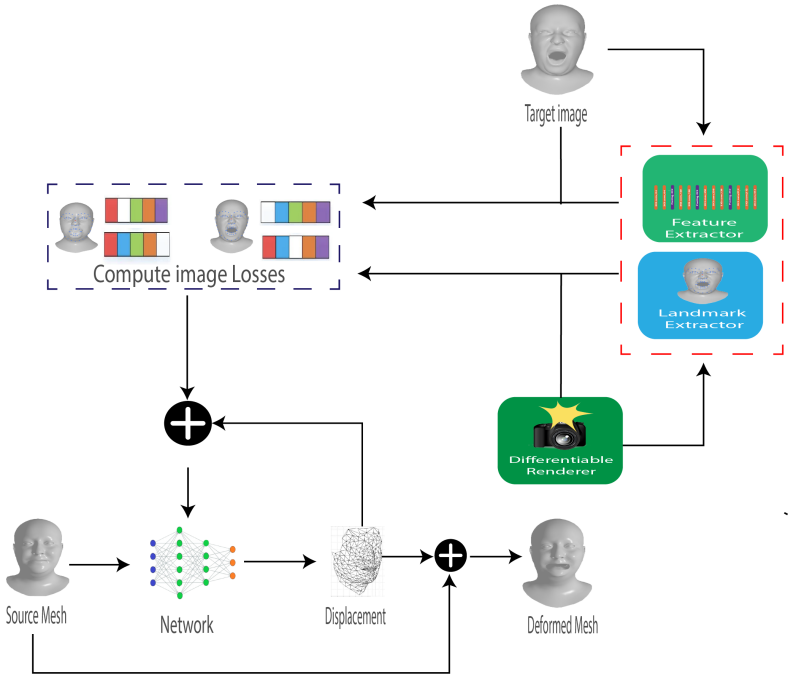


Figure 1: High-Level view of our architecture to deform a 3D mesh through 2D guidance

with the target expression, we extracted both facial landmarks and high and low level features from target image and rendered deformed mesh. After that, their features are compared and their distance used as loss. Although the aforesaid approach is highly generalizable, allowing to work with no spatial information, it also implies challenging aspects to take into account. After rasterization some information like depth are inevitably lost (feedback of the deformation is partially observable), moreover it is not so intuitive to understand in which direction a vertex must be moved in order to have the desired effect in 2D image. The explicit design of our framework can be visualized in figure 1.

3.2. Differentiable Landmark detection

Facial Landmarks are significant points in anatomy, and they are commonly used to analyze some expression features like emotions. A visualization of the extraction of landmarks can be seen in Figure 2. For Landmark detection on image we have used the pre-trained model developed in [8] that is based on the architecture XceptionNet [17]. The original implementation, however, was not suitable for optimization, since relied on non-differentiable operations. The main difficulties arose from the fact that many required pre and post process functions, that were typically implemented through operations or conversions from libraries such as numpy or OpenCV, do not support the gradient tracking. We modified the framework to allow gradient flow in order to perform backpropagation.

As far as we know, there are no available tools or architectures that allow differentiable extraction of 2D landmarks for optimization, which highlights the novelty of our approach. Our method is distinguished by the other Deep Learning approaches since, being not 2D-guided and dealing only with 3D structures, they typically rely on 3D landmarks, and it differs from Geometric Approaches because they work only on tuned face types, require user input to select landmarks or need manual calibration. The original model extracted the Cartesian coordinates of 68 landmarks. For our purpose, we considered only 42 landmarks corresponding to points highlighting delimitations of mouth, eyes and eyebrows as these are the most relevant for expressing emotions. The extracted landmarks were lately used for the computations of two specific loss functions: the distance between each landmark on the rendered mesh and its corresponding landmark on the target face, and the pairwise relative distances among all landmarks on a single face.

$$\mathcal{L}_{\text{landmarks}} = \frac{1}{N} \sum_{i=1}^{N} \left\| l_i^{\text{rendered}} - l_i^{\text{target}} \right\|_2 \tag{1}$$

$$\mathcal{L}_{\text{relative}} = \frac{1}{M}\sum_{i=1}^{N}\sum_{j=i+1}^{N}\left|\left\|l_i^{\text{rendered}} - l_j^{\text{rendered}}\right\|_2 - \left\|l_i^{\text{target}} - l_j^{\text{target}}\right\|_2\right| \tag{2}$$

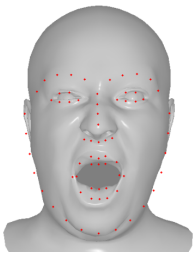


Figure 2: Example of landmarks in an open-mouth expression

3.3. Feature extractor

Similarly to Johnson et al. [12], as mentioned in Chapter 2.1, we have used as feature extractor a VGG-16 network pretrained for classification on ImageNet Dataset [7, 18]. Although the classification task is based on objects, the intermediate layers have learned weights to distinguish specific features, from low-level to high level features. As we can see in figure 3, the network can be naturally divided in five blocks delimited by the pooling stages; we take as feature maps the output of the first four blocks. We are not interested on the output of the fifth block as capturing overly high-level features. We refer to the feature maps extracted from an image I by a block i with $\phi_i(I)$. At the end of this process we have four distinct embeddings for both rendered image and the target image. These embeddings are successively employed in the computation of the Perceptual Loss, that is defined as the weighted sum of the mean squared error of each pair of features extracted from the same block:

$$\mathcal{L}_{\text{Perceptual}} = \sum_{i=1}^{4} w_i \cdot \text{MSE}\left(\phi_i(I^{\text{rendered}}), \phi_i(I^{\text{target}})\right) \tag{3}$$

This loss encourages the network to deform the mesh to match the the target expression's characteristics, such as edges, lineaments, curves and wrinkles.

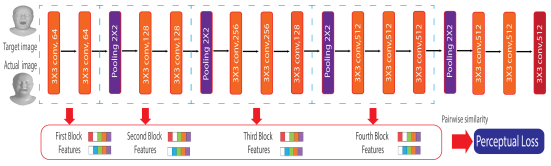


Figure 3: Visualization of the pipeline for the extraction of features from rendered mesh and target image.

3.4. Other Losses

In this section we are going to illustrate additional loss functions used during our proposed pipeline, motivating their employment.

- **Pixel Loss** (L_{mse}): this loss is defined as the mean squared error between pixels of target image and the image rendered from the mesh. It encourages the network to deform the source mesh to match shapes of the target, in particular, adapting to contours and adjusting colors, including also emulating the correct light camera reflection.

$$\mathcal{L}_{\text{mse}} = \text{MSE}(I^{\text{rendered}}, I^{\text{target}}) \tag{4}$$

- **Regularization loss** (L_{reg}): this loss penalizes the magnitude of the displacements produced by the network. It discourages unnecessary or excessive deformations, helping the system to focus on aligning the mesh towards the most relevant features in the target image.

$$\mathcal{L}_{\text{reg}} = \frac{1}{N}\sum_{i=1}^{N}\|d_i\|_2 \tag{5}$$

Depending on the desired kind of alignment, we varied the combination of the losses and the topology of the network. We will talk extensively about this in paragraphs 3.5 and 3.6.

3.5. Network architectures

For our purposes, we have implemented two networks: a Fully connected network (referred to from now as MLP) and a Residual Network. Each network takes in input a vector of size $\mathbf{N} \times \mathbf{3}$ (being the mesh formed by vertices $V \in \mathbb{R}^{n\times 3}$) and returns a vector of size $\mathbf{N}$. In other words, the network takes as input all the vertices of the source mesh and it returns as output a displacement for each vertex to be applied in the direction of its corresponding normal vector. We observed empirically that this implementation was preferable with respect to use a vector of displacements for each vertex along x, y and z directions. Notably, being zero-shot-learning, a major advantage of our networks is that they can accept arbitrary number of vertices. We considered fully connected networks suitable for our task, since they implicitly encode the complete topology of the input mesh structure. In fact, even NSF, which inspired our approach, uses a fully connected network; however, we removed the positional encoding part, as it was sometimes associated with unsatisfactory outputs. The MLP network is used for learning how to move facial keypoint regions. Instead, the Residual network is used in the procedure of feature alignment. This differentiation is due to the fact

we have seen experimentally that in the landmarks alignment process MLP is faster and achieves similar results with respect to the Residual while, for feature alignment, the Residual Network produces smoother and better solutions. Both networks are made up of 4 layers of width 256, and at the end of each layer there is a ReLU activation function. Additionally, each hidden layer is initialized through He initialization: this ensures faster convergence to the minimum of the loss function. At the beginning of the training, the weights of the last layer are all sets to zero, so that at the first iteration the network produces no displacement. From the previous statement follows that initially, the outputted mesh correspond to the input and the network learns gradually how to deform it. At each successive iteration, we also check if the deformation is not such that landmarks can be no more computed, in such case we reset again the weights of last layer.

3.6. Deformation Pipelines

As mentioned in the introduction, we decided to address the expression transfer task differently depending on the kind of the target expression to match:

1. **Moderate Facial Expression (MFE)**: when the target image shows an expression that is close to the neutral one, it is sufficient to optimize the lines of the muscles;
2. **Exaggerated Facial Expression (EFE)**: when the target image exposes a more complex or exaggerated expression, source mesh is passed to an initial processing phase in which the facial keypoints are moved, followed by a phase of optimization of muscles lines for further refinement.

For each of the two objectives described above, we have defined two different pipelines which involve different set of networks, optimization steps and losses.

3.6.1. Pipeline for MFE

The pipeline for Moderate Facial Expressions is composed of a single optimization process; a Residual Network is used to generate displacements to match facial features of the target image. This procedure is guided by a loss defined in the following way:

$$\mathcal{L} = w_{mse} \cdot L_{mse} + w_{per} \cdot L_{perceptual} + w_{reg} \cdot L_{reg} \quad (6)$$

3.6.2. Pipeline for EFE

The pipeline for Exaggerated Facial Expressions is composed by three different optimization steps, in which everytime a new Zero-Shot process is started, considering as input the output of the previous stage (for the first step the input is the source mesh).

1. **First step**: Initially MLP is used to perform some initial keypoint alignment movements, guided by the loss defined below:

$$\mathcal{L} = w_l \cdot L_{landmark} + w_{reg} \cdot L_{reg} \quad (7)$$

2. **Second step**: the output obtained from the first step is used as input for this second passage, and the process described in the Pipeline for MFE 3.6.1 is applied.
3. **Third step** (optional): the output of the previous step passes through MLP for keypoint alignment refinements. The loss is defined again as in equation 7.

4. Results

This section exposes the outcomes of our trials, focusing on expression transfer both when the source mesh and target image depict the same individual and when they differ. For each of the two considered situations we reported an example of MFE and EFE. We took samples from the dataset curated by Ranjan et. al.[15]; it contains high resolution 3D scans of 12 individuals with labeled expressions. As source meshes we used individuals in a neutral expression, while the target images were rendered from faces exhibiting different expressions within the same dataset. This provides a further explanation of why, as stated in the introduction, when both source mesh and target image represent the same individual, the task becomes more similar to reconstructing a target mesh. To be even more clear, the target mesh represents how the person in the source mesh would appear if they adopted the expression shown in the rendered frontal view image. As a consequence of the previous statement, for evaluating this particular task we can employ metrics used in object reconstruction. Moreover, for each of the two considered tasks, we include visual examples of deformed meshes for qualitative evaluation. To benchmark the capabilities of our proposed pipelines, we compared them against Text2Mesh and the default implementation of Pytorch3D. Since these frameworks are for general purpose and were not designed for face-specific fine-grained modifications required for expression transfer, testing them on the proposed task originally led to poor results. A visual representation is shown in Figure 4. In order to create a more meaningful comparison and challenge our approach, we did not just test the results of these frameworks in their original form. Instead, we integrated them into our pipelines by replacing our Networks (MLP or Residual Network) with their displacement generators. In particular, we employed Neural Style Field (NSF) from Text2Mesh and direct geometric optimization of PyTorch3D. From now on these two approaches will become our baselines. In addition to this, to ensure a fair

comparison, we empirically tuned the weights of the single component of the loss functions based on performed experiments in order to achieve the best possible performances.

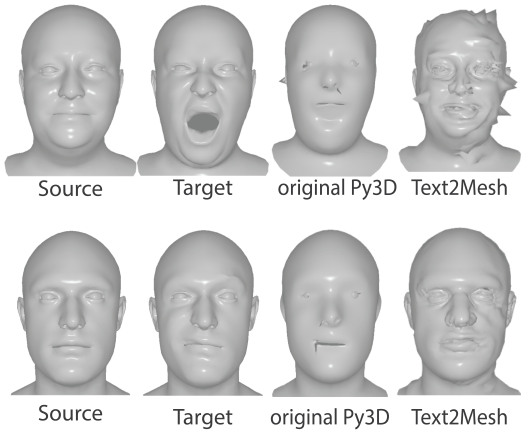


Figure 4: Results of plain PyTorch3D and Text2Mesh frameworks without modifications in the implementation, for expression transfer task.

4.1. Evaluation Metrics

In order to provide a quantitative comparison between our framework and the baselines, in the expression transfer task between the same individual, we employed three widely recognized metrics for measuring similarity in shapes between objects. As said our choice was motivated by the employment of these three measures in some notable works in the field of object reconstruction; examples include PointNet [19], PointNet++ [20] and Deep Closest Point [21]. Although these metrics were primarily designed for point clouds implementations of 3D structures, are still valid for meshes, since it is always possible pass to a point cloud implementation sampling points from meshes.

- **Chamfer Distance**: Chamfer distance computes the average distance between the closest points in two sets. It is effective for capturing overall geometric similarity [22, 9].
- **Hausdorff Distance**: Hausdorff distance computes the greatest distance from a point in one set to the nearest point in another set. It ensures that the maximum distance between the two sets is minimized and it is particularly useful to detect outliers [10].
- **Earth Mover's Distance (EMD)**: Earth Mover's Distance measures the cost to transform one distribution to another. In this case this metric is employed to measure the distance between two point clouds treated as probability distributions. This is mainly employed to capture fine-grained details [11].

4.2. MFE

Figure 5 shows the task of MFE transfer within the same subject, at left the source in neutral expression and at right the target facial gesture of a man raising his eyebrows; we can see that his muscles are visibly in tension and he has a frowning pose. In figure 6 results of different displacement generators are showed. NSF (at left) completely failed in the task as produced an inflated face. By contrast, PyTorch3D (center) and our Framework (right) seem to show graphically similar results, however, as we can see in table 2, Pytorch3D baseline required more optimization steps (36 times the time of our framework) to achieve comparable results. From table 1 we can infer that our framework achieves quantitatively best numerical results across considered metrics. Moreover, we can note that the characteristics of the target expression were successfully passed. In particular:

- Right cheek raised;
- An increase in tension in the temple muscles;
- Nose slightly rotated to the right with minor tension of nasal muscles;
- Change of the light reflection of glabella;
- A distinctive lineament in the upper-right edge of the mouth.

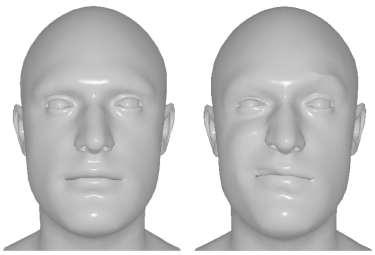


Figure 5: Example of source mesh and target expression for MFE within the same subject.

Table 1
Metrics for MFE transfer within the same subject.

Technique	Chamfer	Hausdorff	EMD
NSF	0.0988	0.0842	0.0974
Py3D	0.0251	0.0423	**0.0273**
Our Fr.	**0.0244**	**0.0403**	0.0306

Additionally, in Figure 7 are reported the visual results of our framework for MFE transfer on different subjects.

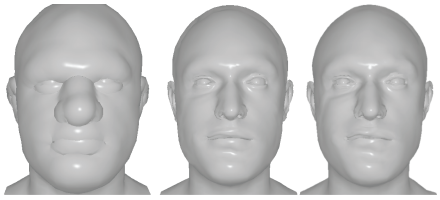


Figure 6: Outputted deformations for MFE within the same subject for NSF (left), pytorch3D (center) and our framework (right).

Table 2
Execution time in minutes across different deformation systems to reach best result on Colab T4 GPU.

Technique	NSF	Py3D	Our Fr.
Time	FAIL	6.41	**0.12**

As we can see, the most important features described above are transmitted, maintaining at the same time the individuality of the source mesh.

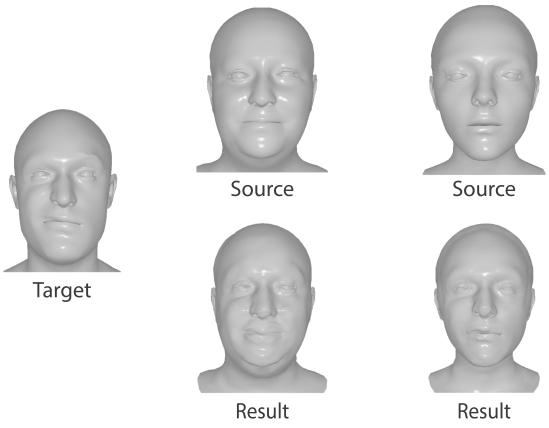


Figure 7: Overview of the produced meshes for MFE transfer on different subjects.

4.3. EFE

Figure 8 shows the task of EFE transfer within the same subject. At left the source in neutral expression and at right the target facial gesture of a man opening his mouth. In this case more computation is required as we need first to move the landmarks of the mouth to suit the necessary deformation. In figure 10, results of different displacement generators, at the end of each main step described in chapter 3.6.2 are showed. Although all three generated satisfying meshes, NSF (at left) is far away from being realistic. Pytorch3D baseline (center) and our Framework (right) seem to show graphically similar results, but the last performed better in modeling the shape of the lips as the mouth is more clearly recognizable. In addition, our framework achieves quantitatively best numerical results in all the considered metrics, as we can see in table 3. Figure 9 shows the number of iterations of the first EFE optimization step to reach the best output according to loss functions (eq. 7); we can clearly see that our framework reaches best results in very few iterations, while the other displacements generator took more. In Figure 11 are reported the results of our framework for EFE transfer on different subjects. All of them opened the mouth while maintaining at the same time the individuality of the source mesh.

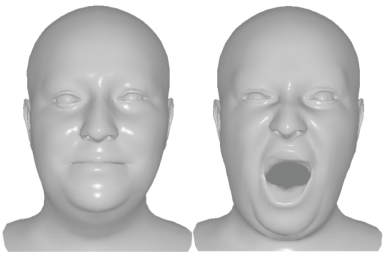


Figure 8: Example of Source mesh and target expression for EFE transfer within the same subject.

Table 3
Metrics for EFE transfer within the same subject, for each of the considered approach.

Technique	Chamfer	Hausdorff	EMD
NSF	0.0385	0.1474	0.0366
Py3D	0.0319	0.1310	0.0379
Our Fr.	**0.0317**	**0.1308**	**0.0365**

5. Conclusion

In this work we proposed a light-weight, fully automatic solution for expression transfer from 2D image to 3D facial meshes. This framework represents an original contribution with respect to the related works in the deep learning field mainly based only on 3D guidance, since it does not require a large dataset and employs original strategies as the 2D differentiable landmark detection. Additionally, even though we are considering only the specific task of expression transfer, our framework consistently outperformed two prominent mesh deformation tools as Pytorch3D and text2Mesh. NSF network from Text2Mesh is surpassed on all metrics and meshes tested, while Pytorch3D implementation delivers weaker results in all or at least in the majority of the metrics analyzed, depending on the mesh considered. This work

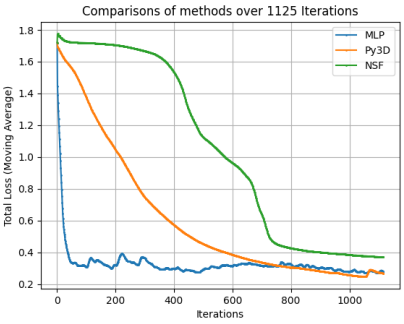


Figure 9: Comparison in number of iterations for each displacement generator for the first EFE optimization stage.

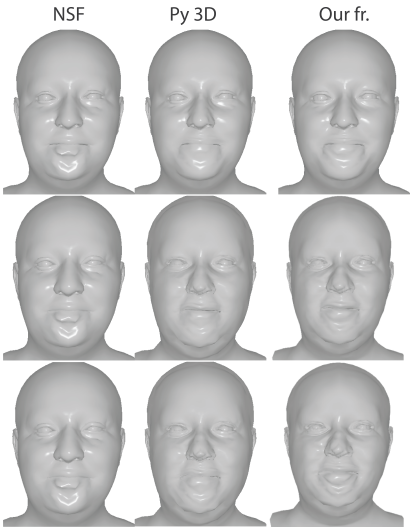


Figure 10: Results at the end of each main stage for EFE transfer within the same subject, for each of the considered approach.

lays the foundations for future extensions, as a matter of facts, we are dealing with a very limited task: we do not handle texture information of the mesh and source and target subjects where captured in similar conditions of light, zoom and angulation. These challenges could be addressed through several extensions, which include trying more complex network architectures, introducing a mapping between the 2D landmarks and specific areas of the 3D face to be deformed and relying on Action Units. With "Action Units" we refer to individual muscle movements responsible for the various facial expressions; as well as done for landmarks, it would be convenient to adapt an existing framework for measuring Action Units in a differentiable way. This would allow the development of a new loss function that can directly provide a

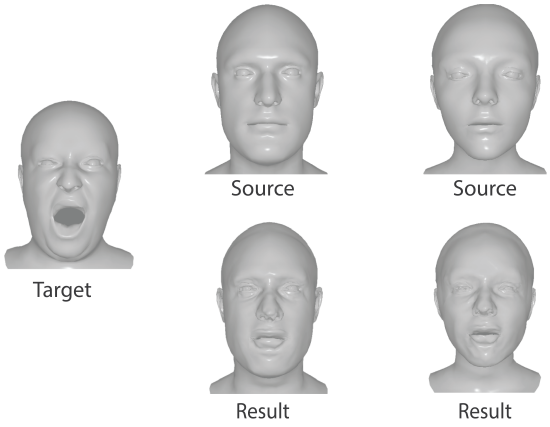


Figure 11: Overview of the produced meshes for EFE transfer on different subjects.

feedback on the deformation of the muscles. In conclusion, with the aforementioned modifications, this work would potentially further improve in outcomes realism and could be applied into broader variety of scenarios.

References

[1] O. Michel, R. Bar-On, R. Liu, S. Benaim, R. Hanocka, Text2mesh: Text-driven neural stylization for meshes, 2021. URL: https://arxiv.org/abs/2112.03221. arXiv:2112.03221.

[2] N. Wang, Y. Zhang, Z. Li, Y. Fu, W. Liu, Y.-G. Jiang, Pixel2mesh: Generating 3d mesh models from single rgb images, in: ECCV, 2018.

[3] H. Onizuka, D. Thomas, H. Uchiyama, R.-i. Taniguchi, Landmark-guided deformation transfer of template facial expressions for automatic generation of avatar blendshapes, in: 2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW), 2019, pp. 2100–2108. doi:10.1109/ICCVW.2019.00265.

[4] C. Ferrari, S. Berretti, P. Pala, A. D. Bimbo, Measuring 3d face deformations from rgb images of expression rehabilitation exercises, Virtual Reality and Intelligent Hardware 4 (2022) 306–323. URL: https://www.sciencedirect.com/science/article/pii/S2096579622000456. doi:https://doi.org/10.1016/j.vrih.2022.05.004, virtual-reality and intelligent hardware in digital twins A).

[5] M. R. Team, Pytorch3d, 2020. URL: https://pytorch3d.org/.

[6] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, I. Sutskever, Learning trans-

ferable visual models from natural language supervision, 2021. URL: https://arxiv.org/abs/2103.00020. arXiv:2103.00020.

[7] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2015. URL: https://arxiv.org/abs/1409.1556. arXiv:1409.1556.

[8] BrainDotAI, Facial-landmarks-detection-pytorch, 2023. URL: https://github.com/braindotai/Facial-Landmarks-Detection-Pytorch, gitHub repository.

[9] T. Wu, L. Pan, J. Zhang, T. Wang, Z. Liu, D. Lin, Density-aware chamfer distance as a comprehensive metric for point cloud completion, CoRR abs/2111.12702 (2021). URL: https://arxiv.org/abs/2111.12702. arXiv:2111.12702.

[10] N. Aspert, D. Santa-Cruz, T. Ebrahimi, Mesh: measuring errors between surfaces using the hausdorff distance, in: Proceedings. IEEE International Conference on Multimedia and Expo, volume 1, 2002, pp. 705–708 vol.1. doi:10.1109/ICME.2002.1035879.

[11] J. M. Solomon, R. M. Rustamov, L. J. Guibas, A. Butscher, Earth mover's distances on discrete surfaces, ACM Transactions on Graphics (TOG) 33 (2014) 1 – 12. URL: https://api.semanticscholar.org/CorpusID:8324821.

[12] J. Johnson, A. Alahi, L. Fei-Fei, Perceptual losses for real-time style transfer and super-resolution, 2016. URL: https://arxiv.org/abs/1603.08155. arXiv:1603.08155.

[13] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, 2020. URL: https://arxiv.org/abs/2006.10739. arXiv:2006.10739.

[14] K. Zou, S. Faisan, B. Yu, S. Valette, H. Seo, 4d facial expression diffusion model, ACM Transactions on Multimedia Computing, Communications, and Applications (2024). URL: http://dx.doi.org/10.1145/3653455. doi:10.1145/3653455.

[15] A. Ranjan, T. Bolkart, S. Sanyal, M. J. Black, Generating 3D faces using convolutional mesh autoencoders, in: European Conference on Computer Vision (ECCV), 2018, pp. 725–741. URL: http://coma.is.tue.mpg.de/.

[16] S. Liu, T. Li, W. Chen, H. Li, Soft rasterizer: A differentiable renderer for image-based 3d reasoning, 2019. URL: https://arxiv.org/abs/1904.01786. arXiv:1904.01786.

[17] F. Chollet, Xception: Deep learning with depthwise separable convolutions, 2017. URL: https://arxiv.org/abs/1610.02357. arXiv:1610.02357.

[18] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, L. Fei-Fei, Imagenet large scale visual recognition challenge, 2015. URL: https://arxiv.org/abs/1409.0575. arXiv:1409.0575.

[19] C. R. Qi, H. Su, K. Mo, L. J. Guibas, Pointnet: Deep learning on point sets for 3d classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE Computer Society, 2017, pp. 652–660. URL: https://doi.org/10.1109/CVPR.2017.16. doi:10.1109/CVPR.2017.16.

[20] C. R. Qi, L. Yi, H. Su, L. J. Guibas, Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5099–5108. URL: https://proceedings.neurips.cc/paper/2017/hash/d8bf84be3800a36b0f72d71842b4c2fc-Abstract.html.

[21] Y. Wang, J. M. Solomon, Deep closest point: Learning representations for point cloud registration, in: Proceedings of the IEEE International Conference on Computer Vision (ICCV), IEEE, 2019, pp. 3523–3532. URL: https://doi.org/10.1109/ICCV.2019.00363. doi:10.1109/ICCV.2019.00363.

[22] A. Hajdu, L. Hajdu, R. Tijdeman, Approximations of the euclidean distance by chamfer distances, CoRR abs/1201.0876 (2012). URL: http://arxiv.org/abs/1201.0876. arXiv:1201.0876.